



Міністерство освіти і науки України
Національний технічний університет України «Київський політехнічний
інститут імені Ігоря Сікорського»
Факультет інформатики та обчислювальної техніки
Кафедра інформаційних систем та технологій

Лабораторна робота №6
З дисципліни «Технології розроблення програмного забезпечення»
Тема: **«ШАБЛОНИ «Abstract Factory», «Factory Method», «Memento»,
«Observer», «Decorator»»**
Система ведення нотаток

Виконав:
Студент групи ІА-24
Шкарніков А. С.

Перевірив:
Мягкий М. Ю.

Київ-2024

Зміст

Тема:.....	3
Мета:	3
Хід роботи	3
1. Реалізувати не менше 3-х класів відповідно до обраної теми	3
2. Реалізувати один з розглянутих шаблонів за обраною темою	5
3. Перевірка патерну	6
Висновки:	7
Код проекту:	7

Тема:

ШАБЛОНИ «Abstract Factory», «Factory Method», «Memento», «Observer», «Decorator»

Мета:

Ознайомитися з основними шаблонами проєктування, такими як «Abstract Factory», «Factory Method», «Memento», «Observer», «Decorator», дослідити їхні принципи роботи та навчитися використовувати для створення гнучкого та масштабованого програмного забезпечення, зокрема у розробці музичного програвача.

Завдання:

В якості нотаток має бути, як мінімум, текст або зображення, додатково можна додати підтримку файлів розміром до 2Мб, збереження посилань на веб-сторінки, збереження html-тексту з коректним його відображенням. Система повинна дозволяти вести нотатки онлайн, заводити блокноти, прикріплювати до нотаток мітки, давати можливість працювати в системі одночасно багатьом користувачам і одному користувачу працювати з різних комп'ютерів. Користувач повинен мати можливість дати доступ до свого блокноту з замітками іншому користувачу з різними правами (тільки перегляд; перегляд та редагування; перегляд, редагування, додавання та видалення нотаток)

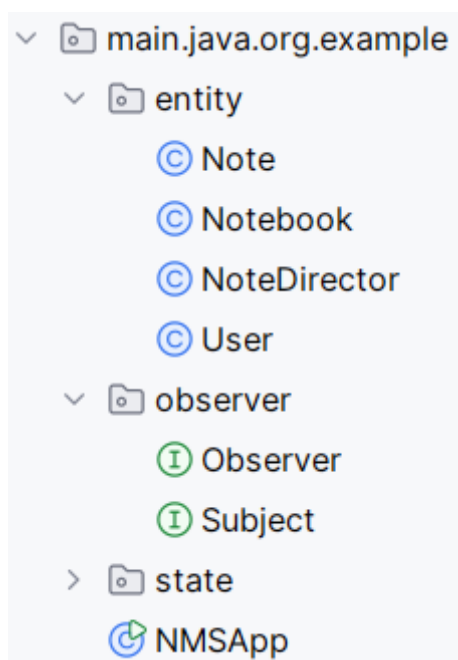
Хід роботи**1. Реалізувати не менше 3-х класів відповідно до обраної теми**

Рис. 1 — Структура проєкту

У процесі виконання лабораторної роботи було створено 2 інтерфейси, реалізовано один клас та оновлено один існуючий, які забезпечують реалізацію функціоналу системи ведення нотаток (рис. 1). Наведемо детальний опис кожного з цих класів:

1. Observer (Інтерфейс):

Клас `MusicPlayer` представляє сам музичний програвач, який відповідає за відтворення, паузу та перемикавання треків у плейлисті. Він взаємодіє з ітератором для доступу до пісень у плейлисті, дозволяючи користувачу керувати відтворенням пісень в залежності від вибраного режиму (наприклад, послідовний, випадковий або повтор). Крім того, цей клас містить методи для збереження та відновлення стану плеєра, що реалізує патерн `Memento`.

2. Subject (Інтерфейс):

Клас `Memento` є внутрішнім класом, який використовується для збереження стану об'єкта `MusicPlayer`. Він інкапсулює всю необхідну інформацію, таку як поточний трек, статус ітератора, позиція в плейлисті та чи знаходиться плеєр на паузі. Це дозволяє зберігати стан плеєра у вигляді об'єкта, який можна пізніше використовувати для відновлення плеєра до його попереднього стану.

3. Notebook (Клас):

Клас `PlayerStateManager` є менеджером стану плеєра, який відповідає за збереження та відновлення станів плеєра, використовуючи об'єкти `Memento`. Він може зберігати кілька різних станів плеєра та відновлювати останній збережений стан за допомогою методів `saveState()` та `restoreState()`. Цей клас дозволяє реалізувати функціональність скасування/повторення операцій або збереження прогресу, щоб плеєр міг відновитися після перезапуску додатку.

4. User (Клас):

`User` представляє користувача системи, який реалізує інтерфейс `Observer`. Кожен об'єкт цього класу отримує повідомлення про зміни в блокноті, до якого він підписаний, через метод `update`. У повідомленнях вказується інформація про додавання, редагування чи видалення нотаток. Цей клас відповідає за обробку отриманих оновлень та їх подання користувачеві, наприклад, через виведення тексту на консоль.

2. Реалізувати один з розглянутих шаблонів за обраною темою

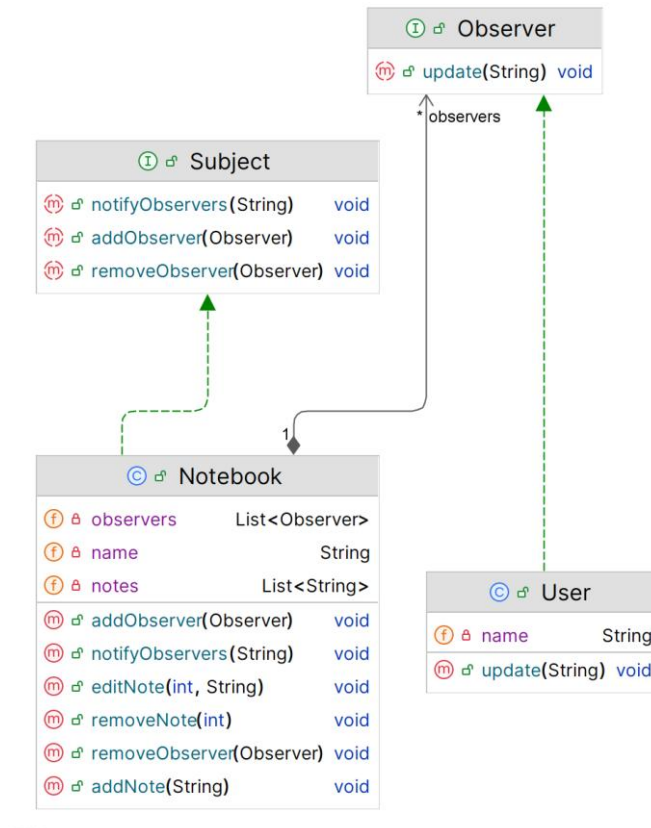


Рис. 2 — Діаграма класів

У системі ведення нотаток патерн Observer було використано для забезпечення автоматичної синхронізації змін у блокноті між різними користувачами, які працюють із ним одночасно. Основна ідея реалізації полягає в тому, що об'єкт блокнота (Notebook) виступає в ролі підписника (Subject), а користувачі (User) — у ролі спостерігачів (Observers).

Коли відбувається будь-яка зміна в блокноті (наприклад, додавання, редагування або видалення нотаток), блокнот повідомляє всіх підписаних користувачів про ці зміни за допомогою виклику їхнього методу `update`. Такий підхід дозволяє всім користувачам отримувати актуальну інформацію про стан блокнота в реальному часі без необхідності вручну перевіряти зміни.

Проблеми, які вирішує патерн

- **Синхронізація даних:** Забезпечує автоматичне інформування всіх зацікавлених сторін про зміни в блокноті, виключаючи можливість виникнення розбіжностей у даних.
- **Підтримка багатокористувацької роботи:** Дозволяє декільком користувачам одночасно працювати з одним блокнотом, отримуючи повідомлення про зміни інших.

- **Мінімізація ручного оновлення:** Користувачі не повинні вручну перевіряти, чи були внесені зміни в блокнот.

Переваги використання

- **Гнучкість:** Нових спостерігачів (користувачів) можна легко додати до системи, не змінюючи логіку блокнота.
- **Зменшення залежностей:** Реалізація підписників і спостерігачів не залежить від конкретних класів, завдяки використанню інтерфейсів Subject і Observer.
- **Масштабованість:** Система легко адаптується до роботи з великою кількістю користувачів.
- **Реальний час:** Патерн забезпечує майже миттєве оновлення стану у всіх спостерігачів, що покращує користувацький досвід.

3. Перевірка патерну

```
public class NMSApp {  
    public static void main(String[] args) {  
        // Create a notebook  
        Notebook notebook = new Notebook( name: "Project Notes");  
  
        // Create users  
        User user1 = new User( name: "Alice");  
        User user2 = new User( name: "Bob");  
  
        // Add users as observers  
        notebook.addObserver(user1);  
        notebook.addObserver(user2);  
  
        // Perform actions on the notebook  
        notebook.addNote("Initial note");  
        notebook.addNote("Another note");  
        notebook.editNote( index: 0, newNote: "Updated initial note");  
        notebook.removeNote( index: 1);  
    }  
}
```

Рис. 3 — Перевірка роботи

У методі main демонструється використання патерна Observer для синхронізації змін у блокноті між кількома користувачами. Спочатку створюється об'єкт блокнота (Notebook) із назвою "Project Notes". Потім створюються два

користувачі — Alice та Bob — які представляють спостерігачів у системі. Далі ці користувачі додаються до списку спостерігачів блокнота, використовуючи метод `addObserver`. Таким чином, будь-які зміни в блокноті автоматично повідомлятимуться цим користувачам.

```
User Alice received update: New note added: Initial note
User Bob received update: New note added: Initial note
User Alice received update: New note added: Another note
User Bob received update: New note added: Another note
User Alice received update: Note edited at index 0: Updated initial note
User Bob received update: Note edited at index 0: Updated initial note
User Alice received update: Note removed: Another note
User Bob received update: Note removed: Another note
```

Рис. 4 — Результат роботи

Після цього в блокнот додаються та редагуються нотатки: створюються дві нові нотатки, перша з яких згодом оновлюється, а друга — видаляється. Кожна з цих змін викликає сповіщення всіх спостерігачів (користувачів) про оновлення. Це дозволяє кожному користувачу автоматично отримувати актуальну інформацію про стан блокнота без додаткових дій з їхнього боку. Код демонструє, як патерн Observer забезпечує узгодженість даних та спрощує багатокористувацьку роботу із блокнотом.

Висновки:

У ході лабораторної роботи було реалізовано патерн Observer у контексті додатку системи ведення нотаток. Це дозволило забезпечити синхронізацію даних між користувачами та спростити взаємодію у додатку.

Код проекту:

https://github.com/Atl4sDev/TRPZ_labs